

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнили:

студенты группы 24ВВВ3

Алмакаев В.В.

Буров А.О.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., Деев М.В.

Пенза 2025

Цель работы – Освоение методов экспериментальной оценки времени выполнения программ на языке C# на примере алгоритма перемножения матриц.

Лабораторное задание:

Задание 1:

1. Вычислить порядок сложности программы (O -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Метод `Random_matrix`:

Два вложенных цикла, каждый до `size`

Количество операций: $\text{size} * \text{size} = \text{size}^2$

Сложность: $O(n^2)$

Метод `multiplication`: Три вложенных цикла, каждый до `size`

Количество операций: $\text{size} * \text{size} * \text{size} = \text{size}^3$

Сложность: $O(n^3)$

Общая сложность программы:

Два вызова `Random_matrix`: $2 * O(n^2)$

Один вызов `multiplication`: $O(n^3)$

При больших n доминирует кубическая сложность

Итоговая сложность: $O(n^3)$

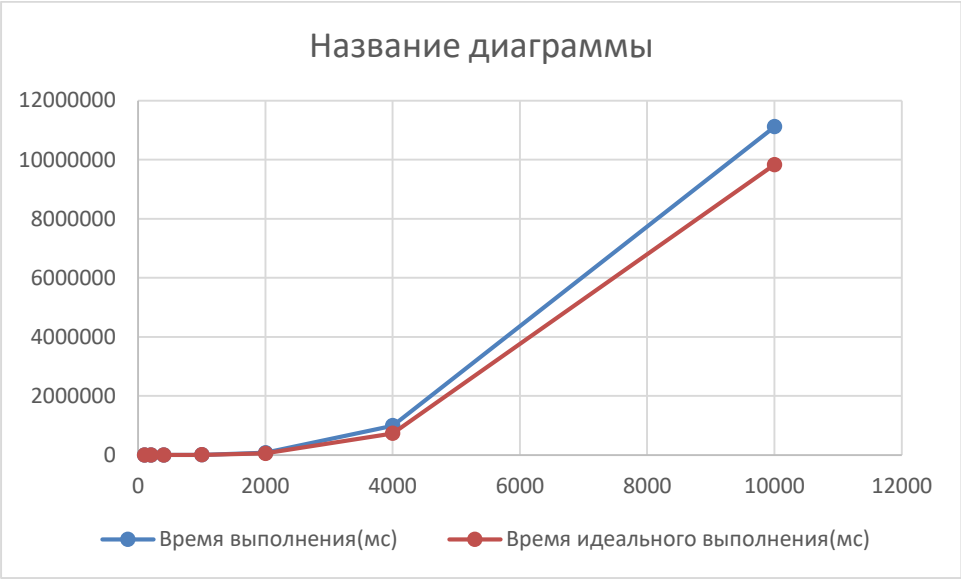
Для программы с размером матрицы 200:

Количество операций умножения: $200^3 = 8,000,000$

Количество операций заполнения: $2 * 200^2 = 80,000$

Таким образом, асимптотическая сложность программы определяется операцией умножения матриц и составляет $O(n^3)$.

Размер матрицы	Время выполнения(мс)
100	5
200	40
400	334
1000	7578
2000	78656
4000	992064
10000	11116416



Код программы

C#

```
using System;

using System.Diagnostics;

using System.Drawing;

using System.Linq;

using System.Net.NetworkInformation;

using System.Collections.Generic;

namespace second
{

    class Program
```

```

{

    static void PrintTableHeader()
    {

        Console.WriteLine("
        _____
        |_____|");

        Console.WriteLine("
        | Размер      | Тип данных | Shell Sort | Quick Sort |
        |_____|
        | Отошение    |
        |_____|");

        Console.WriteLine("
        | массива      |           | (мс)       | (мс)       |
        |_____|
        | QS/Array    |
        |_____|");

        Console.WriteLine("
        |_____|
        |_____|");

    }

    static void PrintTableRow(int size, string dataType, double shellTime, double
    qsTime, double arrayTime)
    {

        double ratio = arrayTime > 0 ? qsTime / arrayTime : 0;

        Console.WriteLine($"
        | {size,-10} | {dataType,-11} | {shellTime,-11:F4} |
        {qsTime,-11:F4} | {arrayTime,-11:F4} | {ratio,-11:F2} |");

    }

    static void PrintTableFooter()
    {

        Console.WriteLine("
        _____
        |_____|");

    }

    static int[,] Random_matrix(int[,] a, int size)
    {

        Random rnd = new Random();

        int i = 0, j = 0;

        while (i < size)

```

```

        {
            while (j < size)
            {
                a[i, j] = rnd.Next(0, 99999);

                j++;
            }

            i++;
        }

        return a;
    }

    static int[,] multiplication(int[,] a, int[,] b, int[,] c, int size)
    {
        for (int i = 0; i < size; i++)
        {
            for (int j = 0; j < size; j++)
            {
                int element_c = 0;

                for (int r = 0; r < size; r++)
                {
                    element_c = element_c + a[i, r] * b[r, j];
                }
            }
        }

        return c;
    }

    static int[] random(int[] a, int size)
    {
        Random rnd = new Random();

        for (int i = 0; i < size - 1; i++)
        {
            a[i] = rnd.Next(0, 99999);
        }
    }

```

```

    }

    return a;
}

static void shell(int[] a)
{
    Stopwatch sw = new Stopwatch();

    sw.Start();

    Random rand = new Random();

    int gap = 1, k, x;

    while (gap < a.Length / 3)
    {
        //gap = (int) (Math.Pow(3, gap) + 1);

        gap = 3 * gap + 1;
    }

    while (gap >= 1)
    {
        // Сортировка вставками с текущим шагом

        for (int i = gap; i < a.Length; i++)
        {
            int temp = a[i];

            int j;

            for (j = i; j >= gap && a[j - gap] > temp; j -= gap)
            {
                a[j] = a[j - gap];
            }

            a[j] = temp;
        }
    }
}

```

```

        gap /= 3;

    }

    sw.Stop();

}

static int[] IncreasingRand(int[] a)
{

    Random rand = new Random();

    a[0] = rand.Next(0, 10);

    for (int i = 1; i < a.Length - 1; i++)
    {

        a[i] = rand.Next(a[i - 1] + 1, 99999 - a.Length + i + 1);

    }

    return a;

}

static int[] DecreaseRand(int[] a)
{

    Random rand = new Random();

    a[0] = rand.Next(90000, 99999);

    for (int i = 1; i < a.Length - 1; i++)
    {

        a[i] = rand.Next(a.Length - i, a[i - 1]);

    }

    return a;

}

```

```

static int[] RandHalf(int[] a)
{

    Random rand = new Random();

    a[0] = rand.Next(0, 10);

    int mid = a.Length / 2;

    for (int i = 1; i < mid; i++)
    {
        a[i] = rand.Next(a[i - 1] + 1, 99999 - a.Length + i + 1);
    }

    for (int i = mid; i < a.Length; i++)
    {
        a[i] = rand.Next(a.Length - i, a[i - 1]);
    }

    return a;
}

```

```

static void qs(int[] a, int left, int right)
{

    int i = left;

    int j = right;

    int pivot = a[(left + right) / 2]; // Выбор опорного элемента (середина
массива)

    do
    {
        while (a[i] < pivot && i < right) i++; // Поиск элемента больше опорного
слева

        while (pivot < a[j] && j > left) j--; // Поиск элемента меньше опорного
справа
    }
}

```



```

        if (i <= j)
        {
            // Обмен элементов

            int temp = a[i];

            a[i] = a[j];

            a[j] = temp;

            i++;

            j--;

        }
    } while (i <= j);

    // Рекурсивные вызовы для подмассивов

    if (left < j) qs(a, left, j);

    if (i < right) qs(a, i, right);

}

static void print(int[] a)
{

    Console.WriteLine("Массив по shell: ");

    for (int i = 0; i < a.Length; i++)
    {

        Console.Write(a[i] + " ");

    }

    Console.WriteLine();

}

static void Main()
{

    for (int i = 1; i < 7; i++)

```

```

{
    if (i == 1) Console.WriteLine("Введите размер матрицы:");
    else Console.WriteLine($"Набор данных: {i}, Введите размер матрицы:");

    int size_for_matrix = Convert.ToInt32(Console.ReadLine());

    int[,] a = new int[size_for_matrix, size_for_matrix];
    int[,] b = new int[size_for_matrix, size_for_matrix];
    int[,] c = new int[size_for_matrix, size_for_matrix];

    Stopwatch sw_for_matrix = new Stopwatch();

    sw_for_matrix.Start();

    Random_matrix(a, size_for_matrix);
    Random_matrix(b, size_for_matrix);

    multiplication(a, b, c, size_for_matrix);

    sw_for_matrix.Stop();

    Console.WriteLine($"Время: {sw_for_matrix.ElapsedMilliseconds}    мс,
размер матрицы: {size_for_matrix}");
}

// Вторая часть с сортировками в виде таблицы
List<int> sizes = new List<int>();

Console.WriteLine("\nВведите 7 размеров массивов для второй части:");
for (int i = 0; i < 7; i++)
{
    sizes.Add(Convert.ToInt32(Console.ReadLine()));
}

PrintTableHeader();

foreach (int size in sizes)
{
    int[] originalArray = new int[size];

    int[] testArray = new int[size];

    Stopwatch sw = new Stopwatch();

```

```
// Тест 1: Случайные данные

random(originalArray, size);

Array.Copy(originalArray, testArray, size);

sw.Restart();

shell(testArray);

sw.Stop();

double shellTime1 = sw.Elapsed.TotalMilliseconds;

Array.Copy(originalArray, testArray, size);

sw.Restart();

qs(testArray, 0, testArray.Length - 1);

sw.Stop();

double qsTime1 = sw.Elapsed.TotalMilliseconds;

Array.Copy(originalArray, testArray, size);

sw.Restart();

Array.Sort(testArray);

sw.Stop();

double arrayTime1 = sw.Elapsed.TotalMilliseconds;

PrintTableRow(size, "Случайный", shellTime1, qsTime1, arrayTime1);

// Тест 2: Возрастающие данные

IncreasingRand(originalArray);

Array.Copy(originalArray, testArray, size);

sw.Restart();

shell(testArray);

sw.Stop();

double shellTime2 = sw.Elapsed.TotalMilliseconds;

Array.Copy(originalArray, testArray, size);

sw.Restart();
```

```

qs(testArray, 0, testArray.Length - 1);

sw.Stop();

double qsTime2 = sw.Elapsed.TotalMilliseconds;


Array.Copy(originalArray, testArray, size);

sw.Restart();

Array.Sort(testArray);

sw.Stop();

double arrayTime2 = sw.Elapsed.TotalMilliseconds;


PrintTableRow(size, "Возрастающий", shellTime2, qsTime2, arrayTime2);


// Тест 3: Убывающие данные
DecreaseRand(originalArray);


Array.Copy(originalArray, testArray, size);

sw.Restart();

shell(testArray);

sw.Stop();

double shellTime3 = sw.Elapsed.TotalMilliseconds;


Array.Copy(originalArray, testArray, size);

sw.Restart();

qs(testArray, 0, testArray.Length - 1);

sw.Stop();

double qsTime3 = sw.Elapsed.TotalMilliseconds;


Array.Copy(originalArray, testArray, size);

sw.Restart();

Array.Sort(testArray);

sw.Stop();

double arrayTime3 = sw.Elapsed.TotalMilliseconds;


PrintTableRow(size, "Убывающий", shellTime3, qsTime3, arrayTime3);

```

```

// Тест 4: Половина отсортирована
RandHalf(originalArray);

Array.Copy(originalArray, testArray, size);

sw.Restart();

shell(testArray);

sw.Stop();

double shellTime4 = sw.Elapsed.TotalMilliseconds;

Array.Copy(originalArray, testArray, size);

sw.Restart();

qs(testArray, 0, testArray.Length - 1);

sw.Stop();

double qsTime4 = sw.Elapsed.TotalMilliseconds;

Array.Copy(originalArray, testArray, size);

sw.Restart();

Array.Sort(testArray);

sw.Stop();

double arrayTime4 = sw.Elapsed.TotalMilliseconds;

PrintTableRow(size, "Половина", shellTime4, qsTime4, arrayTime4);

Console.WriteLine("
|-----|-----|-----|-----|
|-----|");
}

PrintTableFooter();

Console.ReadKey();

}

}

```

Результаты работы программы

```

Введите размер матрицы:
200
Время: 37 мс, размер матрицы: 200
Набор данных: 2, Введите размер матрицы:
100
Время: 4 мс, размер матрицы: 100
Набор данных: 3, Введите размер матрицы:
400
Время: 301 мс, размер матрицы: 400
Набор данных: 4, Введите размер матрицы:
1000
Время: 4656 мс, размер матрицы: 1000
Набор данных: 5, Введите размер матрицы:
500
Время: 561 мс, размер матрицы: 500
Набор данных: 6, Введите размер матрицы:
300
Время: 121 мс, размер матрицы: 300

Введите 7 размеров массивов для второй части:
100
200
300
400
500
600
700
    
```

Размер массива	Тип данных	Shell Sort (мс)	Quick Sort (мс)	Array.Sort (мс)	Отношение QS/Array
100	Случайный	0,1235	0,1075	0,0139	7,73
100	Возрастающий	0,0026	0,0026	0,0019	1,37
100	Убывающий	0,0038	0,0030	0,0026	1,15
100	Половина	0,0045	0,0038	0,0026	1,46
200	Случайный	0,0177	0,0164	0,0064	2,56
200	Возрастающий	0,0070	0,0063	0,0027	2,33
200	Убывающий	0,0077	0,0040	0,0022	1,82
200	Половина	0,0069	0,0068	0,0031	2,19
300	Случайный	0,0231	0,0201	0,0074	2,72
300	Возрастающий	0,0072	0,0073	0,0026	2,81
300	Убывающий	0,0090	0,0058	0,0027	2,15
300	Половина	0,0100	0,0084	0,0038	2,21
400	Случайный	0,0317	0,0269	0,0107	2,51
400	Возрастающий	0,0110	0,0169	0,0044	3,84
400	Убывающий	0,0154	0,0101	0,0040	2,52
400	Половина	0,0168	0,0114	0,0045	2,53
500	Случайный	0,0429	0,0346	0,0138	2,51
500	Возрастающий	0,0112	0,0124	0,0041	3,02
500	Убывающий	0,0161	0,0093	0,0038	2,45
500	Половина	0,0167	0,0157	0,0053	2,96
600	Случайный	0,0520	0,0414	0,0165	2,51
600	Возрастающий	0,0136	0,0144	0,0048	3,00
600	Убывающий	0,0194	0,0119	0,0045	2,64
600	Половина	0,0202	0,0150	0,0064	2,34
700	Случайный	0,0720	0,0517	0,0203	2,55
700	Возрастающий	0,0160	0,0173	0,0062	2,79
700	Убывающий	0,0236	0,0143	0,0056	2,55
700	Половина	0,0217	0,0177	0,0072	2,46

Рисунок 1 - Результат работы программы

Вывод: В ходе данной лабораторной работы были успешно освоены методы экспериментальной оценки времени выполнения программ на языке C#. Была реализована программа, выполняющая перемножение квадратных матриц.